Axiomander

Durable specifications for AI-assisted Python development

Gavin Mendel-Gleason · Scidonia

AI Can (Re)generate Code

It Cannot Infer Intent That Was Never Written

```
# What does this function guarantee?
def reserve(engine, sku, order_id, quantity):
    with engine.begin() as conn:
        row = conn.execute( ... ).fetchone()
        if available < quantity: raise InsufficientStockError
        conn.execute("UPDATE products SET reserved = reserved + :qty  ... ")
    return ReservationReceipt(sku=sku, quantity=quantity)
```

An LLM can rewrite this function in a second. But what was it **supposed** to do? Without a durable specification, the intent is lost the moment the code changes.

Where Behaviour Lives Today

Scattered across:

- Unit tests encode selected examples
- Comments drift from the code
- Tickets describe the original request
- Memory of the original developer

When the code changes:

- Tests may miss the changed behaviour
- Comments become obsolete overnight
- Tickets are closed and forgotten

What we want:

- Intent is explicit and machine-checkable
- The same spec drives testing, checking, and proof
- When code changes, the spec doesn't
- AI-assisted rewrites are validated against the spec

> Axiomander creates a **specification asset** — Gherkin features, contracts, and a domain declaration — that can test and prove many different implementations. The code becomes disposable.

One Specification, Several Uses

Runtime Checking

pre/post conditions checked on every execution

Scenario Testing

every Gherkin Examples row exercised against the contract

AI Guidance

regenerated code validated against the existing spec

Documentation

the contract IS the behavioural documentation

Counterexamples

mismatches between impl and spec surface automatically

Machine Proof

universal properties checked by the Rocq proof kernel

Running Example: Inventory Reserve

An ordinary Python function against SQLAlchemy:

```
def reserve(self, engine, sku, order_id, quantity):
    with engine.begin() as conn:
        row = conn.execute(
            text("SELECT on_hand, reserved FROM products"
                 " WHERE sku = :sku"), {"sku": sku}
        ).fetchone()

        if row is None: raise ProductNotFoundError(sku)

        on_hand, reserved = row
        available = on_hand - reserved

        if available < quantity:
            raise InsufficientStockError(sku, quantity, available)

        conn.execute(
            text("UPDATE products SET reserved = reserved + :qty"
                 " WHERE sku = :sku"),
            {"qty": quantity, "sku": sku},
        )

    return ReservationReceipt(sku=sku, quantity=quantity)
```

The Same Behaviour as a Contract

```
reserve_contract = Contract(
    requires=[
        lambda s, a: a.quantity > 0,              # valid input
        lambda s, a: a.sku in s.products,         # product exists
    ],
    ensures=[
        lambda s, a, r, s2: (                     # what must hold after
            s2.products[a.sku].reserved
            == s.products[a.sku].reserved + a.quantity),
    ],
    exceptions=[
        ExcExit(raises=InsufficientStockError,
                when=[lambda s, a:
                    s.products[a.sku].on_hand
                  - s.products[a.sku].reserved < a.quantity]),
    ],
    writes={"state.products[sku].reserved"},
    reads={"state.products[sku].on_hand",
           "state.products[sku].reserved"},
)
```

One spec. Tested on real data. Lowered to proof obligations.

What Each Clause Means

requires — valid inputs and starting state

checked before every call; rejected scenarios fail here

ensures — what must be true afterwards

checked after every successful call against the post-state

exceptions — expected failure cases

"when this condition holds, this error must be raised"

writes / reads — what may change, what may be consulted

everything else is checked unchanged — no manual "unchanged" clauses

The contract language is a **restricted, side-effect-free fragment of Python**. Predicates cannot modify state, call I/O, or raise exceptions — so they are safe to execute, translate, and prove.

Run One Scenario

A Gherkin row: sku=S1, on_hand=100, reserved=10, quantity=30

1. Materialize

Turn the row into a temp SQLite database and function arguments

2. Pre-check

Take a snapshot of the database state; check **requires**

3. Execute

Run the real `reserve()` function against SQLAlchemy on the temp database

4. Frame check

Verify only `products[S1].reserved` changed — nothing else did

5. Derived check

Recompute totals from the snapshot; confirm they are consistent

6. Post-check

Take a second snapshot; check **ensures** and invariants

Frames Catch Unintended Changes

The contract declares what may change:

```
writes = {"state.products[sku].reserved"}
```

Axiomander checks that:

- `reserved` for the selected product changed correctly
- `on_hand` for that product is unchanged
- every other product row is untouched
- other tables and logs are unchanged

Why this matters.

Without frame checking, a refactor can silently mutate bystander state — a different product, an unrelated table, a log channel — and no test will catch it unless it was specifically written for that row.

The frame is **derived** from the write declaration. You never write "everything else unchanged."

When Contract and Implementation Disagree

The contract says:

```
reserved increases by 30
```

The implementation does:

```
reserved increases by 40
```

Result: a concrete counterexample is surfaced.

Either the code is wrong, or the contract is wrong.

The scenario exposes the disagreement as a concrete failure — a specific row with specific values. The developer resolves it: fix the code, or fix the contract.

Specification is iterative. Contracts are not written once and frozen. They are refined as edge cases are discovered, as the domain is understood better, as gaps are found. Each cycle tightens the spec.

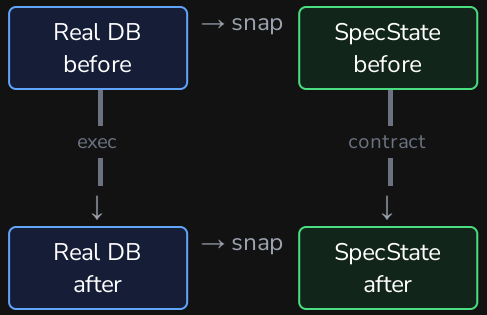
Once proved, the contract guards refactoring. With machine-checked proof that the abstract model satisfies the contract, the implementation can be rewritten — new library, new schema, new language — and the same spec still tells you what behaviours must obtain.

The Same State Model for Testing and Proof

Before execution, Axiomander reads the concrete system into a specification state.

After execution, it applies the same observation function again.

The contract compares those two states.



Testing and proof share the same declared state structure.

At runtime: the left path — `observe`, execute, `observe` — runs against real SQLite. The contract predicates evaluate to Python bool on real state.

At proof time: the right edge — `contract` — is not a function but a **predicate** over (pre-state, args, result, post-state). It comes from the contract's `ensures` clause. The lowering proves that for every admissible pre-state there exists a post-state satisfying the predicate.

Symmetry binds them: the same `observe` is used before and after execution, so the concrete left path and the abstract right path reason about the same interpretation of state.

Why a Tested Model Beats a Mock

The Problem with Mocks

Ordinary mocks simulate library behaviour but have no verified semantics. They may silently accept unsupported operations or return different results from the real library.

Axiomander's Approach: Differential Testing

Each library theory (SQLAlchemy, logging, OpenTelemetry) ships with a **differential fidelity suite** — the same program runs against the real library and against the theory model. Results must agree on the supported fragment.

Covered operations
Stub and real library produce identical results — verified by the suite.

Out-of-fragment
Rejected loudly — no silent approximation of unsupported behaviour.

State model
Operations interpreted over a pure, inspectable state model — not a mock.

From Python Predicate to Machine-Checked Theorem

```
Python contract → mathematical propositions → Rocq proof kernel → verdict
```

PROVED
machine-checked by the kernel

UNKNOWN
queued for AI-assisted proof

COUNTEREXAMPLE
concrete failure found at runtime

The AI may propose proof scripts — but only the Rocq kernel validates them. Untrusted output is never accepted merely because a model produced it.

Two Adoption Paths

Adopt Around Existing Code

- Feature — Gherkin scenario tables
- Implementation — write the production code
- Testing (sanity) — make sure it basically works
- **Contract** — capture observed behaviour as a spec
- Testing (contract) — re-run under contract checking
- Formal proof — lower to Rocq

Start with the code you have. Write contracts that describe what it already does. The contract validates existing behaviour.

Build from the Contract

- Feature — Gherkin scenario tables
- **Contract** — write spec as acceptance criteria
- Implementation — build code against the spec
- Testing — validate the implementation
- Formal proof — machine-checked guarantee

Write the spec first. Use it as acceptance criteria and AI-implementation guidance. Code becomes replaceable.

Both paths converge on a durable specification that survives code changes.

What Has Been Implemented and Proved

Current prototype status:

Domain	Ops	Rows	Obligations
inventory	3	22	6/6, 6/6, 4/4 proved
bank_transfer	1	11	7/7 proved
Total	**4**	**33**	**23/23 proved**

This demonstrates the end-to-end pipeline at prototype scale.

What's trusted, what's proved:

Proved: store-obligation consistency, frame soundness, invariant preservation — checked by the Rocq kernel.

Trusted: the projection observes the concrete system faithfully; the library theory conforms to the real library behaviour.

Ongoing: trace/event obligations, larger-scale evaluation, counterexample surfacing from the runner into the scoreboard.

Generated Code Is Replaceable

The Specification and Its Evidence Survive

Without a durable specification

code is rewritten and intent is lost

With Axiomander

implementation can be replaced; specification, tests, and proof evidence remain